ChatGPT

基于 Python 的雀魂式立直麻将单人对战小游戏实施方案

执行摘要

本方案面向“单人对战 + 其余全为 AI”的日本立直麻将（包含三麻与四麻）小游戏，目标是在功能与规则层面尽可能复刻雀魂的规则体验，并以 **Python + MySQL 8.0** 为核心技术约束，给出可落地的工程化实施路线。规则层面的关键点包括：四麻与三麻的差异（如三麻去除万子 2–8、无吃、北拔作宝牌等）、途中流局（四风连打、四杠散了、四家立直、九种九牌）、流局满贯（Nagashi Mangan）、多家荣和与“无三家和了流局”等雀魂特性，以及立直棒/供托棒与本场棒结算等细节。官方可直接引用的雀魂赛事房间规则参数（例如赤宝牌数量、食断、供托点数、本场点数、途中流局开关、三响流局开关等）可参考其赛事手册中给出的“Game rules”配置项。[1]

工程实现上，建议采用“**服务端权威牌局内核 + 事件溯源存档**”的设计：所有摸打鸣立/翻宝/结算由后端核心状态机驱动，前端仅提交意图并显示结果；同时将“动作序列（Action Log）+ 随机种子（Seed）+ 关键状态快照（Snapshot）”落库，实现回放、断线重连、对局审计和 AI 训练数据复用。规则判断与番符计分建议复用成熟的开源日麻计算库（如 MahjongRepository 的 `mahjong`），其宣称可计算役种、符、番与点数，并在大量天凤牌谱上验证过一致性，有助于显著降低“役判/符判/点表”实现风险。[2]

AI 方面，建议按难度分层：从“规则/向听数驱动的启发式 AI”逐步迭代到“策略网络 + 价值网络”的学习型 AI。最强难度如要逼近“联动大教练/高水平教学 AI”的强度，需要引入监督学习与自博弈强化学习的组合路线：可参考 Suphx 在日麻场景中使用深度强化学习、引入全局回报预测与运行时策略自适应等手段的研究思路；[3] 三麻方向可参考面向 Sanma 的深度强化学习工作（如 Meowjong），其明确提出三麻与四麻在牌山/动作集合（Chii 被 Kita 替代）等方面的差异，并采用“多动作模型 + 自博弈强化学习微调”的路径。[4] 高吞吐模拟训练可参考 Mjx 这类为日麻 AI 研究设计、强调高性能模拟与研究友好 API 的框架思想。[5]

另外，合规层面需要强调：雀魂的文本、美术、音频、角色与客户端资源受著作权与协议约束，开发时应避免复制其资源与专有内容（可用开源牌面素材或自行绘制），并在产品叙述上避免造成官方授权或同人合作的误导；同时可参考其用户协议中关于知识产权与打击外挂/未授权程序的条款精神来设计日志、校验与反作弊策略。[6]

功能清单与优先级

下表按 **MVP / 次要 / 可选** 分层，目的是让你一个人也能分阶段交付：先做“可玩 + 规则正确 + 能存档回放”，再做“体验完善 + AI 变强”。

模块	MVP（必须）	次要（建议）	可选（增强/长期）
账号与本地进度	单机用户（本地注册/登录/游客）、基础统计（对局数、平均顺位）	多角色档位、成就系统	云同步、多设备登录

模块	MVP（必须）	次要（建议）	可选（增强/长期）
模式与房间	四麻：东风/半庄；三麻：东风/半庄；单人对局固定“1 人 + N AI”	规则预设切换（段位场风格/赛事房风格/友人场可选项集）	自定义房规 UI（更多开关）、特殊玩法房（后期）
牌局核心	发牌、摸打、吃碰杠、立直、和牌（荣和/自摸）、牌山与岭上牌、宝牌/里宝/杠宝、听牌判定、振听（含临时振听）、流局与连庄	动画时序更贴近（如鸣牌/翻宝展示时机）、更细的规则兼容（如部分边界规则开关）	教学提示（可解释“为何不能荣和/为何振听/为何无役”）
流局体系	荒牌流局 + 听牌罚符 + 供托/本场处理；途中流局：九种九牌/四风连打/四家立直/四杠散了（按规则开关）[7]	流局满贯（Nagashi Mangan）[7]	更完整的“规则差异包”导入（不同赛事/平台差异）
役种与计分	覆盖雀魂常规役种、役满/双倍役满、符计算、点数计算、立直棒与本场棒结算	可选古役（local yaku）整体开关[8]	自定义古役、役种讲解与可视化
AI 难度	L1-L3：基础可玩（向听/进张/安全牌/简单攻守）	L4：强 AI（启发式 + 蒙特卡洛/近似期望值）	L5：学习型“教练级”模型（监督学习 + 自博弈强化学习）[9]
回放/存档/重连	完整 Action Log 回放、对局存档、断线重连（恢复到最近快照）	对局导出（JSON/压缩包）、关键节点书签（立直/和牌/流局）	“牌谱分享码”、局面编辑器（用于 AI 训练与调试）
排行榜与统计	基础排行榜（胜场、平均顺位、最大连胜）	Elo/Glicko 之类的强度评分（对 AI 级别与用户）	牌谱分析（放铳率/立直率/副露率等）
运维与质量	日志、错误上报、自动化测试、数据备份	性能剖析面板、回归测试自动跑 AI 对战	多平台发行、增量更新系统

系统架构与技术选型理由

总体架构

建议采用“**客户端界面 + 后端牌局服务 + AI 决策服务 + MySQL 持久化**”的分层，即便你当下只做单机，也仍按“本地部署的 C/S”组织代码。这样做的直接收益是：
1) 规则与随机性更容易做到“权威、可复现、可审计”；
2) 回放/重连天然落在后端事件流上；
3) AI 训练与对战跑批可以复用同一套牌局内核，不被 UI 牵制。

```
flowchart LR
  UI[前端/界面\n桌面或Web] -->|WebSocket/HTTP| API[后端API\n牌局编排]
  API --> CORE[牌局核心引擎\n状态机+规则校验]
  CORE --> SCORE[番符与点数模块\n役判/符判/点表]
```

```
CORE --> RNG[随机与牌山模块\nseed->墙->王牌]
CORE <--> AI[AI决策模块\n多难度]
API --> DB[(MySQL 8.0\n用户/对局/动作/回放)]
CORE --> LOG[日志/指标\n调试与审计]
```

以上结构的关键是：把“**状态机（合法动作集合）**”作为单一真相来源，任何 UI/AI 只是在“当前可行动作集合”中选择。此模式与赛事房间规则参数的“开关化配置”兼容度高（例如途中流局开关、三响流局开关、食断/赤宝牌数量等）。[10]

后端与核心引擎

核心引擎建议拆成五个可独立测试的子模块：

- **State（状态）**：完整局面数据结构（手牌、河、鸣牌、副露、立直状态、宝牌指示牌、供托、本场、剩余牌数、玩家点数等）。
- **Rules（规则）**：从 `State` 推导合法动作集合，并提供 `validate(action, state)` 与 `apply(action, state)`。
- **Flow（流程）**：实现“谁行动—是否进入响应窗口—如何裁决多家响应—何时结算—何时开新局”的状态机。
- **Scoring（计分）**：对“和牌形/役/番符/点数/责任支付”进行计算与结算；多家和牌时的供托归属与支付顺序必须严格一致（雀魂支持多家荣和，且三响不按流局处理）。[11]
- **Event Sourcing（事件溯源）**：每次动作落一条不可变事件（Action Log），可选定期落“快照”加速重连与回放。

在役判与点数方面，强烈建议复用成熟库：例如 MahjongRepository 的 `mahjong`（Python）在功能描述中包含“役种/符/番/得点计算、和牌校验、向听数计算”，并宣称在大量天凤牌谱上验证一致性，可显著降低“计分实现不一致”的风险。[2]

AI 模块

AI 模块需要同时满足两类需求：
1) **在线对局实时决策**（低延迟、稳定、可解释）
2) **离线训练/评估**（高吞吐、可并行、可回放对照）

因此建议 AI 模块提供两套接口：

- `act(observation, legal_actions, budget_ms) -> action`：对局实时决策
- `selfplay(n_games, ruleset, seed) -> replays`：自动生成训练数据与评估数据

当进入“最强教练级”阶段，建议引入深度学习框架与训练库：Tianshou 作为 PyTorch DRL 库有较完整算法与统一接口，并在 JMLR 有论文介绍其模块化与训练基础设施特点。[12]

数据库与数据结构策略

MySQL 8.0 主要用于：用户数据、对局结果、动作日志、AI 配置版本、排行榜与统计聚合。对“规则配置/动作 payload/快照元数据”等结构化但频繁演进的字段，建议使用 `JSON` 列；如需对 JSON 内的关键字段做索引，可用 **生成列（generated column）+ 索引** 的方式实现间接索引，这在 MySQL 官方文档中属于推荐范式。[13]

前端/界面

你一个人开发，界面建议优先“**信息清晰 + 操作顺畅**”，而非高质量 3D/粒子特效。可选两条路线：

- **桌面客户端（推荐单人开发）**：Qt（PySide6）或类似 GUI，优势是部署简单、无需浏览器适配、与本地后端同机运行。
- **Web 前端**：浏览器 UI + WebSocket，优势是跨平台与后期扩展；但会引入更多前端工程量。

无论选哪条，都建议 UI 与规则引擎之间只通过“状态快照 + 合法动作列表”交互，避免 UI 内产生任何“私自推演规则”的分叉逻辑。

部署形态

- **本地开发/运行**：Docker Compose（后端 + MySQL）+ 本地 UI。
- **云端可选**：单人对战若考虑“云端排行榜/牌谱分享”，则后端与数据库上云，客户端只做 UI；但要接受“单人也要维护服务”的运维成本。

MySQL 8.0 数据库表设计

设计原则

1) **事件溯源优先**：动作日志是回放与审计的根；局面快照是优化手段而非真相来源。
2) **可复现**：每局/每小局记录随机种子与规则版本，保证“同一动作序列可复盘到同一结果”。
3) **AI 可追责**：AI 的每次决策应记录“模型版本/参数/随机温度/搜索深度”等，便于回归与对比评测。
4) **统计可聚合**：排行榜与统计用“汇总表 + 物化更新”或“定时任务重算”均可，但必须能从原始对局数据重建。

表结构总览

下面给出满足你要求的核心表（用户、牌局、手牌/操作记录、AI 配置、排行榜等），并补充规则集与快照以支撑“回放/重连/审计”。

users（用户）

字段	类型	约束	说明
user_id	BIGINT UNSIGNED	PK	自增
username	VARCHAR(32)	UNIQUE	显示名
password_hash	VARBINARY(64)		推荐 Argon2/bcrypt 结果
created_at	DATETIME(3)	IDX	创建时间
last_login_at	DATETIME(3)		最近登录
status	TINYINT		0 正常 / 1 封禁 / 2 删除

索引：`UNIQUE(username)`；`INDEX(created_at)`。

rulesets（规则集版本）

字段	类型	约束	说明
ruleset_id	BIGINT UNSIGNED	PK	
name	VARCHAR(64)		如 “4P_RANKED_LIKE”
mode	ENUM('4P','3P')	IDX	三麻/四麻
config_json	JSON		规则开关与参数（赤宝牌数、途中流局开关、三响流局、kuitan 等）
engine_version	VARCHAR(32)		牌局引擎版本号
created_at	DATETIME(3)	IDX	

索引：`INDEX(mode, created_at)`。

ai_models（AI 模型/算法版本）

字段	类型	约束	说明
ai_model_id	BIGINT UNSIGNED	PK	
name	VARCHAR(64)		“Heuristic_v1”、“CoachNet_v0.3”
kind	ENUM('HEURISTIC','SL','RL')	IDX	启发式/监督/强化
artifact_uri	VARCHAR(255)		模型文件路径（本地/对象存储）
sha256	CHAR(64)	UNIQUE	文件摘要
metadata_json	JSON		网络结构、训练集、超参等
created_at	DATETIME(3)	IDX	

ai_profiles（AI 难度配置）

字段	类型	约束	说明
ai_profile_id	BIGINT UNSIGNED	PK	
display_name	VARCHAR(32)		“简单/普通/困难/教练”
level	TINYINT	IDX	1-5
ai_model_id	BIGINT UNSIGNED	FK->ai_models	绑定模型
params_json	JSON		温度、搜索深度、风险系数等
created_at	DATETIME(3)		

games（牌局：一场东风/半庄）

字段	类型	约束	说明
game_id	BIGINT UNSIGNED	PK	
owner_user_id	BIGINT UNSIGNED	FK->users	玩家
mode	ENUM('4P','3P')	IDX	
ruleset_id	BIGINT UNSIGNED	FK->rulesets	
seed	BIGINT UNSIGNED		全局随机种子
status	ENUM('RUNNING','FINISHED','ABORTED')	IDX	
started_at	DATETIME(3)	IDX	
finished_at	DATETIME(3)		
client_version	VARCHAR(32)		UI/客户端版本（用于兼容）

game_players（牌局参与者：1 人 + N AI）

字段	类型	约束	说明
game_player_id	BIGINT UNSIGNED	PK	
game_id	BIGINT UNSIGNED	FK->games	IDX
seat	TINYINT	IDX	0=东/1=南/2=西/3=北（3P 不用北）
player_type	ENUM('HUMAN','AI')	IDX	
user_id	BIGINT UNSIGNED	NULL/FK->users	人类玩家
ai_profile_id	BIGINT UNSIGNED	NULL/FK->ai_profiles	AI
start_points	INT		起始点
end_points	INT		终局点
placement	TINYINT		1/2/3/4

联合索引：`INDEX(game_id, seat)`；并约束同一 game 内 seat 唯一。

rounds（小局：东1局、东2局…）

字段	类型	约束	说明
round_id	BIGINT UNSIGNED	PK	
game_id	BIGINT UNSIGNED	FK->games	IDX

字段	类型	约束	说明
round_no	SMALLINT	IDX	例如1..（含延长）
prevalent_wind	ENUM('E','S','W')		场风（含西入）
dealer_seat	TINYINT		当前庄位
honba	SMALLINT		本场数
riichi_sticks	SMALLINT		场上立直供托数
wall_seed	BIGINT UNSIGNED		小局随机种子（可从game seed派生）
init_dora_indicators	JSON		初始宝牌指示牌（136编码或34编码）
status	ENUM('RUNNING','ENDED')	IDX	
ended_reason	ENUM('TSUMO','RON','DRAW','ABORTIVE','NAGASHI_MANGAN')		
ended_at	DATETIME(3)		

actions（操作记录：摸/切/鸣/立/和/流局等）

字段	类型
action_id	BIGINT UNSIGNED
round_id	BIGINT UNSIGNED
seq	INT UNSIGNED
actor_seat	TINYINT
action_type	ENUM('DRAW','DISCARD','CHI','PON','KAN','KITA','RIICHI','RON','TSUMO','PASS','REVEAL_DORA','DRAW_END
tile_136	SMALLINT
payload_json	JSON
state_hash_pre	CHAR(64)
state_hash_post	CHAR(64)
created_at	DATETIME(3)

联合索引：`INDEX(round_id, seq)`；`INDEX(action_type, created_at)`。

round_settlements（结算明细）

字段	类型	约束	说明
settlement_id	BIGINT UNSIGNED	PK	
round_id	BIGINT UNSIGNED	FK->rounds	UNIQUE
result_json	JSON		winners/losers、番符、役列表、符明细、点数分配、本场与供托处理、包牌等
created_at	DATETIME(3)		

state_snapshots（快照：断线重连/回放加速）

字段	类型	约束	说明
snapshot_id	BIGINT UNSIGNED	PK	
round_id	BIGINT UNSIGNED	FK->rounds	IDX
seq	INT UNSIGNED		对应 action seq
snapshot_blob	LONGBLOB		压缩后的完整状态（建议 msgpack+zstd）
created_at	DATETIME(3)		

leaderboards（排行榜/统计聚合）

字段	类型	约束	说明
leaderboard_id	BIGINT UNSIGNED	PK	
mode	ENUM('4P','3P')	IDX	
ruleset_id	BIGINT UNSIGNED	FK->rulesets	IDX
user_id	BIGINT UNSIGNED	FK->users	IDX
games_played	INT UNSIGNED		
avg_placement	DECIMAL(4,3)		平均顺位
rating	INT		可选 Elo
updated_at	DATETIME(3)	IDX	

示例数据

以下示例用于说明字段形态（并非强制 SQL 风格）：

```
INSERT INTO users(user_id, username, password_hash, created_at, status)
VALUES (1001, 'player01', X'AA55...', '2026-04-14 10:00:00.000', 0);
```

```sql
INSERT INTO rulesets(ruleset_id, name, mode, config_json, engine_version, created_at)
VALUES (
 2001, '4P_RANKED_LIKE', '4P',
 JSON_OBJECT('aka_dora', 3, 'open_tanyao', TRUE, 'honba_points', 300, 'triple_ron_draw', FALSE),
 'engine_0.1.0', '2026-04-14 10:00:00.000'
);

INSERT INTO ai_models(ai_model_id, name, kind, artifact_uri, sha256, created_at)
VALUES (3001, 'Heuristic_v1', 'HEURISTIC', 'local://models/heur_v1.json', REPEAT('0',64),
'2026-04-14 10:00:00.000');

INSERT INTO ai_profiles(ai_profile_id, display_name, level, ai_model_id, params_json, created_at)
VALUES (4001, '普通', 2, 3001, JSON_OBJECT('risk', 0.5, 'defense', 0.5), '2026-04-14 10:00:00.000');

INSERT INTO games(game_id, owner_user_id, mode, ruleset_id, seed, status, started_at)
VALUES (5001, 1001, '4P', 2001, 987654321, 'RUNNING', '2026-04-14 10:05:00.000');
```

ER 关系图

```mermaid
erDiagram
  USERS ||--o{ GAMES : owns
  RULESETS ||--o{ GAMES : uses
  GAMES ||--o{ GAME_PLAYERS : has
  AI_MODELS ||--o{ AI_PROFILES : provides
  AI_PROFILES ||--o{ GAME_PLAYERS : assigned
  GAMES ||--o{ ROUNDS : contains
  ROUNDS ||--o{ ACTIONS : records
  ROUNDS ||--|| ROUND_SETTLEMENTS : settles
  ROUNDS ||--o{ STATE_SNAPSHOTS : snapshots
  USERS ||--o{ LEADERBOARDS : ranked_in
  RULESETS ||--o{ LEADERBOARDS : scoped_by
```

规则与计分映射及实现要点

本节把“雀魂规则需求”拆成可实施的规则条目，并给出实现要点与边界情况。你可以把它当成“规则验收清单”。

规则预设与关键参数

建议至少实现两套预设：

- **四麻：Ranked-like 预设**（模拟段位场常用体验）
- 允许食断（Open Tanyao/ Kuitan） [14]
- 赤宝牌数量：3（红五万/筒/索各 1） [14]
- 立直点棒：1000；本场棒：每本场 300（荣和放铳者 +300×本场；自摸每家 +100×本场） [15]
- 宝牌：表宝、杠宝、里宝、杠里宝开启 [16]
- 途中流局：四风连打/四杠散了/四家立直/九种九牌开启 [7]

- 三家和了流局（Sanchahou）：关闭（即三响不作流局处理） [7]
- 流局满贯（Nagashi Mangan）：开启 [7]

- **三麻：雀魂 Sanma 预设**

- 去除万子 2–8（仅留 1m 与 9m），且 Mixed Triple Sequence（Sanshoku Doujun）不可达 [17]
- 不可吃（Chii disabled） [17]
- 北为拔北宝牌（Kita/Nukidora），且其行为会影响一发、九种九牌、两立直等（见下） [17]
- 本场点：每本场 200；不听罚符：2000 [17]
- 自摸得点：默认自摸损（Tsumo-loss），友人场可选“北家折半”（North-Bisection） [18]

牌山与摸打流程

1) **牌组构成**
四麻标准为 136 张牌（34 种 × 4），EMA 规则明确此常规模型，并说明花牌等不用于立直麻将。 [19]
三麻为 108 张（27 种 × 4），并且动作集合包含 Kita；Meowjong 对此给出明确描述。 [20]

2) **王牌与岭上**

- 王牌区（dead wall）常规为 14 张；杠后从岭上摸牌并翻杠宝。EMA 对“王牌区与最后一摸/最后一打”的处理有详细描述。 [21]
- 雀魂赛事房中明确开启“Kan Dora/Ura Dora/Kan-Ura Dora”，并可配置“明杠立即翻宝牌”开关；手册截图显示“明杠立即翻宝牌：Disable”。 [10]
- 实现建议：将“翻杠宝时机”作为规则参数（`kan_dora_reveal_timing`），并在事件流中显式记录 `REVEAL_DORA` 动作，保证回放一致。

3) **鸣牌优先级与响应窗口**

- EMA 规则中有“claim precedence and timing”条目（虽该截图未展开全文，但目录与相关章节说明了优先级是规则的一部分）。 [22]
- 工程实现上建议使用“响应窗口（reaction window）”机制：任何一次 `DISCARD` 之后，系统根据规则计算每个对手的可响应集合（RON/PON/CHI/KAN/PASS），再按优先级裁决并推进状态机。多家荣和时则可能产生多个赢家。

4) **多家荣和与供托归属**

- 雀魂支持多家荣和（Multiple rons: Available），且“立直供托归属：由放铳者之后顺位最近的赢家获得（atamahane）”。 [23]
- 同时官方赛事房规则明确“三响流局：No”，与“无三家和了流局”一致。 [7]
- 实现要点：
- 结算必须支持 `winners[]`（长度 1~3）以及对每个赢家分别计算其役/番符/点数。
- 供托（立直棒）在多家和牌时归属“头跳赢家”，其余赢家只拿自己牌的点数收支；这一点需要在 `round_settlements.result_json` 中明确记录，以便回放与统计不歧义。 [24]

立直、振听与里宝

1) **立直基本规则与供托**
EMA 对立直宣告、缴纳 1000 点棒、流局后供托累积与多家赢家时供托归属有明确文字说明。 [21]

- 实现要点：

- 立直动作应拆成两步：`RIICHI_DECLARE`（宣言 + 供托扣点）与随后的 `DISCARD`（立直宣告牌），并把“横置的舍牌”作为 UI 表现而非规则数据。
- 里宝仅在“立直或两立直成立且赢家为该立直宣告者”时才可翻开并计番。 [25]

2) **振听（含临时振听）**
EMA 对振听与临时振听有明确定义：若能用自己弃牌中的等待牌和牌，则不得荣和；临时振听在“错过荣和机会”后持续到下一次摸牌/鸣牌（立直后则持续到局终）。 [26]

- 实现要点：
- `furiten_permanent`：历史弃牌集合命中当前等待集合。
- `furiten_temp`：在一次响应窗口中“可荣和但选择 PASS”后置位，直到该玩家下一次 `DRAW` 或 `CALL` 清除（立直状态则不清）。
- 校验：`RON` 动作必须检查 `furiten_*` 与“是否有役”两个维度（允许后付 atozuke）。 [27]

荒牌流局、罚符、本场与连庄

1) **荒牌流局与听牌罚符（noten penalty）**
EMA 明确：荒牌流局后听牌者展示手牌；总罚符为 3000 点，并按听牌人数分配（1 听：+3000；2 听：各 +1500；3 听：各 +1000）。 [21]

- 雀魂赛事房中“Noten Penalty”作为参数出现（其数值可能随赛事房规则而变），因此工程上应将罚符表作为可配置项而非写死。 [10]
- 三麻默认罚符总额为 2000（No-ten penalty: 2000），这是雀魂三麻规则差异点之一。 [17]

2) **本场（honba/counters）**
EMA 说明：东家和牌或荒牌流局后放置本场棒；每本场使和牌点数增加 300（自摸每家 +100，荣和放铳者 +300）。 [28]

- 雀魂赛事房同样给出 “Continuance Counter Points: 300”。 [10]
- 三麻雀魂规则中本场点为 200（Honba point: 200）。 [17]
- 实现要点：将 `honba_points_ron`、`honba_points_tsumo_each` 作为规则参数，避免三麻/四麻硬编码分叉。

3) **连庄（renchan）**
雀魂规则总结中指出：多家荣和时若庄家在赢家集合中仍连庄，否则轮庄；流局满贯则以庄家是否听牌决定连庄等。 [23]

- 实现要点：
- 计算 `dealer_holds` 的逻辑必须以“结果类型 + 庄家状态”作为输入，而不是简单地看是否有人和牌。
- 由于雀魂支持延长战（enchousen），终局判定还要考虑“目标点数/1位必要点数”“All Last 条件”等平台规则差异；雀魂规则总结明确“Extended match / Extra wind round: Applied”。 [29]

途中流局与流局满贯

1) **途中流局开关集合**
雀魂赛事房明确存在四风连打/四杠散了/四家立直/九种九牌开关。 [10]
三麻因缺少北家，雀魂规则总结指出不具备“四风连打/四家立直”流局。 [17]

- 实现要点：
- 途中流局检测属于“动作后检查（post-action check）”，例如“四风连打”需在第一巡且无人鸣牌且四家首打同风成立后触发。

- `ABORTIVE_END` 事件要进入结算逻辑：通常不结算罚符，但本场与供托棒处理要按规则参数来（建议显式建模）。

2) **流局满贯（Nagashi Mangan）**
雀魂赛事房明确开启 “Mangan at Draw: Yes”。[10] riichi.wiki 的雀魂条目将其作为可计分的特殊结果列出。[8]
EMA 规则对流局满贯的定义存在（其文中描述：弃牌全为幺九字牌且未被鸣走等）。[30]

- 实现要点：
- 在荒牌流局判定阶段，先检查是否有人满足流局满贯；若成立则 **不进行听牌罚符交换**，改按“自摸满贯”方式结算（雀魂也采用类似处理思路）。[31]
- 需要记录“是否有人鸣走该玩家任何一枚弃牌”的布尔状态（可在 action 流中实时更新）。

役种清单与逐条实现映射

你需求是“雀魂全部役种（立直麻将）”，建议按“役种枚举 + 单元测试牌例”驱动开发。riichi.wiki 对雀魂常规役种表与可选 local yaku 给出了完整列表与若干交互限制（例如：一发/海底不可与岭上复合；国士可抢暗杠；九莲不可抢暗杠；责任支付适用于大三元与大四喜等）。[8]

常规役种（建议作为必须实现集合）

番数层级	役种（示例）	主要判定输入	关键边界点
1番	立直、门前清自摸、平和、一杯口、一发、断幺九、役牌（场风/自风/三元）、抢杠、岭上、河底/海底	是否门前、是否自摸、和牌形分解、事件类型（杠/最后一张/立直后 1 巡）	一发会被鸣牌/杠/北拔等打断；里宝只在立直成立时翻[32]
2番	两立直、七对子、混全带幺九、一气通贯、三色同顺、三色同刻、三槓子、对对和、三暗刻、小三元、混老头	和牌形分解（七对/面子）、副露情况、牌种组成	部分役开门降番（如一气/三色/混全）需参数化[33]
3番	二杯口、纯全带幺九、混一色	同上	二杯口必须门前；纯全/混一色开门降番[34]
6番	清一色	同上	开门降番；与宝牌叠加形成累计役满时的上限规则需配置[35]
满贯特殊	流局满贯	弃牌序列、是否被鸣走、是否门前等	与罚符互斥，且连庄规则要一致[36]
役满/双倍役满	天和/地和、四暗刻/单骑、国士/十三面、九莲/纯正、四杠子、字一色、绿一色、大三元、小四喜/大四喜等	起手/第一巡事件、和牌形分解、等待形、杠次数、牌种组成	责任支付（包牌）适用于大三元与大四喜；国士可抢暗杠但九莲不可[8]

可选古役（local yaku）
雀魂条目指出 local yaku 在段位场默认关闭，但在部分模式可开启，且是“全开或全关”，不可单独选取。[8] 若你要“完整规则覆盖”，建议在次要阶段实现，并以 `ruleset.config_json.local_yaku_enabled` 控制。

实现建议（强烈落地化）

- 役判实现不建议“纯手写 if-else 乱长”；推荐采用：
  1) **和牌形分解层**：输出标准化 block（顺子/刻子/杠/雀头/等待形/副露来源）
  2) **役判层**：每个役一个纯函数，输入为“分解 + 局面事件上下文（立直、杠、海底等）”
  3) **聚合层**：按规则求最大番（同一手牌多分解取最高价值），EMA 也明确“若有多种计分方式取最高”。[28]

- 若使用 `mahjong` 库：以它的役判/符判为基础，再补齐“雀魂差异项”（例如三麻 nukidora、自摸损/北折半、明杠翻宝时机、多家和牌供托归属等）。[37]

牌局流程与状态机设计

总体流程图

```
flowchart TD
  A[GameInit\n加载ruleset/座位/起始点] --> B[RoundInit\n生成墙/王牌/初始牌姿]
  B --> C[Deal\n发牌: 庄14 其余13]
  C --> D[TurnLoop\n当前行动者]
  D --> E{阶段}
  E -->|摸牌阶段| F[Draw\n抓牌/岭上补牌]
  F --> G[ActionSelect\n可选动作集合]
  G --> H{动作类型}
  H -->|自摸/立直/暗杠/加杠/拔北/切牌| I[ApplySelfAction]
  I --> J[Discard\n若产生舍牌]
  J --> K[ReactionWindow\n对手响应: ron/pon/chi/kan/pass]
  K --> L{裁决}
  L -->|一人或多人荣和| M[WinResolve\n逐赢家计分/供托归属/本场加点]
  L -->|鸣牌成立| N[CallResolve\n移牌/更新副露/更新轮次]
  L -->|无人响应| O[NextTurn\n下家摸牌]
  N --> D
  O --> D
  D --> P{流局检测}
  P -->|荒牌流局| Q[DrawEnd\n罚符/供托/本场/轮庄]
  P -->|途中流局| R[AbortiveEnd\n按ruleset处理]
  P -->|流局满贯| S[NagashiMangan\n按满贯结算]
  M --> T[RoundEnd\n更新round/points/honba]
  Q --> T
  R --> T
  S --> T
  T --> U{GameEnd?}
  U -->|否| B
  U -->|是| V[GameFinal\n终局结算/排行榜写入]
```

该流程与雀魂“开启途中流局、关闭三响流局、开启流局满贯”等配置相匹配。[7]

状态定义、输入/输出与校验规则

下表给出你实现时最需要“写清楚并单测”的状态机契约（每个状态只接受特定输入，否则拒绝并记录异常日志）。

状态	允许输入（动作）	主要输出	校验要点（失败即拒绝）
GameInit	创建牌局	写 games/game_players	ruleset 存在；座位数量与 mode 一致（3P 禁用北位） [17]
RoundInit	系统事件	rounds 初始化、dora 指示牌确定	wall 构造正确；3P 去除万子 2–8；初始牌河清空 [17]
Deal	系统发牌	各家手牌、庄家 14 张	张数校验；不可出现重复牌；发牌顺序仅影响“可复现”，不影响规则结果
Draw	系统摸牌	手牌 +1，剩余牌数 -1	若剩余牌数为 0，进入流局判定；杠后摸岭上牌需标记事件用于“岭上开花”役判 [38]
ActionSelect	人/AI 提交意图	进入 Apply	动作必须在 `legal_actions(state)` 集合中；立直需门前且听牌；立直供托足额（通常 1000） [25]
Discard	人/AI 切牌	生成舍牌事件	切牌必须来自手牌；立直后切牌必须是宣言时指定或符合规则约束；更新“临时振听”候选 [39]
ReactionWindow	系统生成响应集合	进入裁决	对每家分别计算：是否可荣和（含役/振听校验）、是否可碰/吃/杠（3P 禁吃） [40]
WinResolve	赢家集合（1~3）	round_settlements 写入、点棒变化	多家荣和需分别计分；供托归属“放铳者之后最近赢家”；三响不流局 [41]
CallResolve	pon/chi/kan/kita	更新副露与行动权	Kuikae（食替）在雀魂规则中“不允许”，需校验鸣牌后立即切回限制牌 [42]；kita 会打断一发/两立直/九种九牌等（雀魂三麻规则）。 [17]
DrawEnd	荒牌流局	罚符结算、本场+1、供托留桌	罚符总额按规则（4P 常见 3000；3P 2000）；立直者必须展示听牌否则视为 noten（并可能触发处罚逻辑） [32]
AbortiveEnd	途中流局	按开关处理本场/连庄/供托	仅在对应条件满足触发（四杠散了等开关）；3P 不存在四风连打/四家立直流局 [43]
NagashiMangan	流局满贯	按“自摸满贯”结算并处理连庄	弃牌全幺九字且未被鸣走；与罚符互斥 [44]
GameFinal	终局判定	games/game_players 更新、leaderboards 更新	延长战（enchousen）与“未达目标点数继续”等规则需按 ruleset 参数实现；剩余供托归属规则（雀魂：剩余供托归 1 位） [29]

AI 体系设计与性能预算

难度分层目标

你的约束是“单人开发”，所以 AI 方案必须“**渐进可用**”：先有能打完一局且不明显乱打的 AI，再逐步增强到“可教学/可对抗”的水平。

建议难度分 5 级（L1-L5）：

等级	定位	核心技术	可解释性	预计开发成本
L1 简单	新手陪练	规则合法 + 随机/固定策略（优先进张）	高	低
L2 普通	可玩对手	向听数驱动：最小向听、最大进张（ukeire）+ 基础防守（现物/筋）	中高	中
L3 困难	接近网麻中低段	启发式期望值：立直/副露决策、危险度估计（放铳率模型）+ 局面策略（点差/局次）	中	中高
L4 强敌	高段对手	蒙特卡洛抽样/有限信息推演（rollout）+ 对手模型（副露倾向/立直倾向）	中低	高
L5 教练级	“大教练”目标	监督学习策略网络 + 自博弈强化学习（policy gradient/PPO 等）+ 离线分析器（推荐手/失误解释）	可通过解释器补足	极高

这里的“L5 教练级”可以以学术工作为实现路线参照：Suphx 在日麻中采用深度强化学习，并引入全局回报预测、oracle guiding、运行时策略自适应等技术点，是达到高水平的典型路线之一。[3] 三麻方向可参考 Meowjong：其将 Sanma 的动作拆成 discard/pon/kan/kita/riichi 五个模型，并在 discard 上通过自博弈强化学习增强。[4]

训练/调参方法与数据需求

L1-L3（不依赖机器学习的可交付路线）

1) **向听数与进张计算**

- 需要快速计算：`shanten(hand)`、`ukeire(hand)`、`waits(hand)`。开源 `mahjong` 库包含向听与和牌校验能力，可用于快速落地。[45]

2) **攻守权衡（启发式）**

- 进攻得分近似：`EV_attack ≈ P(win) * expected_points - P(deal_in) * expected_loss`。
- 防守危险度估计：采用经验规则（现物 0%、筋、壁、字牌危险度随巡目变化）+ 立直后危险度上调。

3) **三麻特化**

- 三麻手进展更快、打点更大、且存在 North/Kita 动作（会打断一发/两立直等），策略需更激进；Meowjong 也指出三麻相较四麻更偏进攻风格。[20]
- 三麻得点需要支持自摸损与北折半两种结算机制（默认 Ranked 为自摸损，友人场可北折半）。[18]

L4-L5（学习型/教练级路线）

1) **环境与模拟吞吐**
强化学习对样本量要求极高，因此需要高性能模拟器或尽可能快的自研内核。Mjx 论文强调其作为开源日麻框架，核心目标之一是提升模拟速度，并报告其在基准中可达到数量级的加速。[5]
2) **训练数据来源**

- **监督学习（SL）**：可用公开牌谱（需注意许可/协议）或自生成牌谱（用较强启发式 AI 作为 teacher）。
- **自博弈强化学习（RL）**：从 SL 初始化出发，自博弈产生轨迹并用 PPO/PG 等更新策略。Meowjong 的流程是“多动作 CNN 预训练 + discard 模型自博弈 RL 增强”。[4]
  3) **评估指标**

- 局级：和率、自摸率、放铳率、立直率、副露率、平均打点、流局听牌率。

- 场级：平均顺位、1 位率、pt/elo、不同点差局面的策略合理性（All Last 的抉择）。
- 教练级还要加：与“基准 AI”对战的统计显著性、以及“推荐手一致率/后验价值提升”。

可复用算法与模块清单

- **向听/有效牌（ukeire）计算器**：L1-L5 全用。
- **危险度模型（Defence model）**：L2 起用；L3-L5 逐步替换为学习模型。
- **期望值估计器（EV estimator）**：L3-L5 全用。
- **搜索器（rollout / IS-MCTS）**：L4；L5可作为推理时增强（类似运行时策略自适应的“在线修正”思想）。[46]

性能/延迟预算

参考雀魂赛事房对局计时“Thinking Time: 5+20s”这类在线时间配置，可以把在线 AI 推理时间预算设为：

- L1-L3：**50–150ms/决策**（体验顺滑）
- L4：**200–800ms/决策**（可接受轻微思考）
- L5：**100–300ms（神经网络推理）+ 可选 200–1000ms（轻量搜索）**
  赛事房规则截图可作为“目标体验”的参考基线。[1]

单人对战支撑能力：匹配、回放、存档、重连、反作弊、日志调试

单人匹配与开局编排

单人对战本质是“开房间自动补 AI”：

- 输入：模式（3P/4P）、时长（东风/半庄）、AI 难度分配（例如 1 个困难 + 2 个普通）、规则预设。
- 输出：`games`、`game_players`、首局 `rounds` 初始化。

建议做一个“编排器（match orchestrator）”层：它不懂规则细节，只负责选择规则集与 AI 配置、生成种子并启动引擎。

回放与牌谱

核心设计：Action Log=唯一真相

- 每个动作写入 `actions`（seq 严格递增），并存 `state_hash_pre/post`。
- 回放时从 `RoundInit` 开始重放动作序列即可复原全局。
- 关键点：所有“系统动作”也必须记录（发牌、翻宝牌、流局结算），避免回放时依赖隐含逻辑。

导出格式建议

- JSON Lines（每行一个 action）+ 头部 metadata（ruleset/seed/版本）。
- 若后期想做 AI 训练，可直接把回放转成“(observation, legal_actions, chosen_action, reward)”样本。

存档与断线重连

断线重连对单机也有意义（崩溃恢复/意外退出）：

- 每 N 条 action（例如 20 条）落一个 `state_snapshots`：压缩后的完整状态。
- 重连：加载最后快照 + 重放其后的 action。
- 校验：重放后 `state_hash_post` 必须一致，否则判为存档损坏并触发“回滚到更早快照/只读回放模式”。

作弊防护与审计思路

单人本地对战的作弊动机主要来自“刷排行榜/刷战绩”。完全离线无法绝对防作弊，但可以做到：**篡改成本显著上升 + 审计可发现**。

建议策略：

1) **服务端权威**

- 不信任客户端：客户端只发“我想打这张牌/我想立直”，后端验证后才落 action。
  2) **动作链哈希（hash chaining）**

- `state_hash_post = H(state_hash_pre + action_payload + canonical_state)`，形成不可轻易伪造的链。
  3) **随机性承诺（commit-reveal 可选）**

- 开局先记录 `seed_commit = H(seed)`；终局再把 seed 明文写入结果，防止“看到牌山后改 seed”。
  4) **异常检测**

- 对 AI/玩家决策耗时、非法动作次数、重连频率做统计，触发告警。
  5) **设计对齐行业反作弊原则**

- 雀魂协议中提到会用技术手段识别未授权、危害公平的程序，并可能收集与此相关的信息；虽然你不需要复刻其做法，但可以借鉴“对局日志 + 异常检测 + 处罚/标记”的治理思路。 [6]

日志、调试工具与可观测性

1) **结构化日志**：每条 action 以 JSON 形式输出（包含 game_id/round_id/seq/actor/action_type/耗时/校验结果）。
2) **规则断言**：开发期可打开 `assert_invariants(state)`：例如“牌总数守恒”“手牌张数合法”“副露张数一致”等。
3) **可视化调试面板（建议次要实现）**：

- 显示当前向听、有效牌数、危险度热力图、各 AI 的打牌理由（L2-L3 级别可解释）。
  4) **教练分析（可选）**：

- 雀魂客户端内有分析机器人（MAKA）的描述条目，这说明“对局后评估与给分”属于该产品体验的一部分，你的“教练级难度”也可以拆成“强 AI 对手 + 分析器/讲解器”两条线并行推进。 [47]

开发计划、测试计划、部署运维与成本可行性评估

开发里程碑与单人工时估算

假设你每周可投入约 20–30 小时（业余）或 40 小时（全职），给出两档计划。重点是：**规则正确性与可回放性**先于“AI 很强”。

阶段一：可玩 MVP（建议 8–12 周全职，或 16–24 周业余）

- 周期内主要交付：四麻可完整打完东风/半庄 + 3 个难度 AI + 存档回放 + 基础数据库。
- 风险最高点：役判/符判/多家和牌结算/途中流局边界。

建议周拆解（以全职 10 周为例）：

周期	交付物	关键风险与对策
第1-2周	项目骨架：后端引擎工程、状态结构、动作枚举、事件溯源落库	先定义“状态机契约”，否则后期改动巨大
第3-4周	四麻基本流程：发牌/摸切/鸣牌/立直/和牌校验/流局	先跑通“合法动作集合”，UI 只做显示
第5周	计分接入：番符点数模块 + 本场/供托 + 多家荣和供托归属	用成熟库降低役判风险 [45]
第6周	途中流局/流局满贯/三响不流局	对照赛事房开关与规则总结 [7]
第7周	三麻规则接入（去牌/无吃/拔北/自摸损）	三麻差异多且影响状态机 [48]
第8周	AI L1-L2：向听+进张+基础防守	先“像人”再“变强”
第9周	UI 完整可用、回放/重连、基础统计与排行榜	事件溯源是回放根基
第10周	全量回归测试、修 bug、性能优化	建立 AI 对战回归基准

阶段二：强 AI 与教练能力（3–9 个月，取决于目标强度）

- L3-L4：启发式 EV + sampling/rollout（偏工程 + 领域知识）。

- L5：需要搭建训练流水线、数据、分布式对局跑批与评测；参考 Suphx/Meowjong/Mjx 的研究路线意味着投入会显著上升。 [9]

测试计划

测试应覆盖三个维度：规则正确性、数据一致性、AI 回归稳定性。

1) **单元测试（Unit）**

- 役判测试：每个役至少 5–20 个牌例（含开门/门清、里宝、一发、岭上等边界）。
- 符测试：双风雀头 4 符、平和自摸/荣和符、坎张/边张/单骑等。 [49]
- 流局测试：九种九牌触发条件、四家立直触发条件、三麻不出现四风连打/四家立直。 [43]

2) **集成测试（Integration）**

- “固定 seed + 固定 AI”跑 1000 局，断言结果可复现（action hash 链一致）。
- “随机 seed + 固定 AI”跑 1 万局，断言不崩溃、牌数守恒、无非法状态。

3) **AI 对战回归测试（Regression）**

- 每个 AI 版本在同一组 seed 上对打 N 局，比较关键指标（放铳率/平均顺位/平均打点）是否出现非预期劣化。
- 对 L5：增加“策略漂移”监控（模型更新后与旧模型对局胜率需在合理区间）。

部署方案与运维要点

本地部署（推荐起步）

- Docker Compose：
- `game-server`（Python 后端 + AI 推理）
- `mysql`（MySQL 8.0）
- 可选 `redis`（缓存、任务队列）
- 数据备份：每日 `mysqldump` 或物理备份；对回放数据可按 game_id 分区归档（减少主库膨胀）。

云部署（可选）

若要做“排行榜/分享码/跨设备”，才建议上云：

- 单机规格即可起步（2C4G + SSD）
- 监控：CPU/内存、请求延迟、对局崩溃率、数据库慢查询
- 备份：主从或定期快照；回放大字段（snapshot/blob）建议对象存储分离

成本估算与可行性结论

成本项	MVP（仅启发式 AI）	含 L5 教练级（深度学习）
开发时间（单人）	2–6 个月（取决于投入强度）	6–18 个月（强依赖训练与调参）
服务器	可纯本地	若上线：月均几十美元级别起（轻量服务）
训练算力	无	需要 GPU（至少 1×消费级 GPU，训练/自博弈规模越大越贵）

成本项	MVP（仅启发式 AI）	含 L5 教练级（深度学习）
主要风险	规则边界与计分一致性	样本效率、训练稳定性、算力成本、策略收敛难度

可行性结论：

- **做出规则正确、体验接近、AI 可玩的单机对战版本是可行的**，关键是依赖成熟计分库并用事件溯源保证稳定回放。 [50]
- **做到“教练级强 AI”属于研究级工程**：建议把“强对手 AI”与“对局分析器/讲解器”拆开，以“先提供可解释建议与复盘评分”逐步逼近最终目标。 [51]

开源库、参考资料与优先查阅来源清单

必选/高优先级开源库

- `mahjong`（MahjongRepository）：日麻役种/符/番/点数、和牌校验、向听数计算；并宣称在大量天凤牌谱上验证一致性，适合作为计分与基础分析底座。 [2]
- `mjai` / `mjai.app`：用于 AI 竞赛与模拟的日麻模拟器实现，可借鉴其“模拟器 + bot 提交”思路做离线对战回归。 [52]
- `Mjx`：面向日麻 AI 研究的高性能框架思想参考（即便你不直接用，也建议阅读其“stateless protocol / gym-like API / 性能对比”部分）。 [5]
- `Tianshou`：PyTorch 强化学习库（若走 RL 路线）。 [12]

规则与计分权威资料

- **EMA 立直麻将规则（2016）**：包含立直、振听、罚符、本场、计分流程等清晰条文，可用作规则引擎的“规范化参照”。 [53]
- 雀魂赛事房规则参数截图：用于确认雀魂房规开关（赤宝牌数量、食断、里宝/杠里宝、途中流局开关、三响流局开关等）。 [1]
- riichi.wiki 的雀魂条目：汇总雀魂规则差异（多家荣和、供托归属、途中流局集合、三麻详细规则、自摸损/北折半等）。 [54]

AI 研究优先读物（论文/学术）

- Suphx（深度强化学习日麻 AI）：提供高水平 AI 的核心技术参考点。 [3]
- Meowjong（Sanma 深度强化学习）：专门针对三麻的动作集合与训练流程。 [4]
- Mjx（高吞吐日麻模拟框架）：为训练和评测提供工程参考。 [5]

素材与许可提示

- Meowjong 文中提到其牌面图形资源改编自 CC BY 许可的开源牌面仓库，并在论文中标注许可来源；你同样应使用明确许可的牌面素材或自行绘制，以规避侵权风险。 [55]
- 与雀魂资源相关的知识产权与反作弊条款可在其协议文本中找到表述方向，开发时应避免直接复用其美术/文本/音频与专有资产。 [6]

---

[1] [7] [10] [11] [14] [15] [16] [36] [43] [44] https://mahjongsoul.yo-star.com/tournament/rules.pdf
https://mahjongsoul.yo-star.com/tournament/rules.pdf

2 37 45 50 https://pypi.org/project/mahjong/
https://pypi.org/project/mahjong/

3 9 46 51 Suphx: Mastering Mahjong with Deep Reinforcement ...
https://arxiv.org/abs/2003.13590?utm_source=chatgpt.com

4 20 48 55 https://ieee-cog.org/2022/assets/papers/paper_170.pdf
https://ieee-cog.org/2022/assets/papers/paper_170.pdf

5 https://ieee-cog.org/2022/assets/papers/paper_162.pdf
https://ieee-cog.org/2022/assets/papers/paper_162.pdf

6 fuwuxieyi_chs_t.txt
https://mahjongsoul.game.yo-star.com/v0.8.24.w/docs/chs_t/fuwuxieyi_chs_t.txt?utm_source=chatgpt.com

8 17 18 23 24 27 29 33 34 35 41 42 47 49 54 https://riichi.wiki/Mahjong_Soul
https://riichi.wiki/Mahjong_Soul

12 https://jmlr.org/papers/v23/21-1127.html
https://jmlr.org/papers/v23/21-1127.html

13 https://dev.mysql.com/doc/refman/8.4/en/create-table-secondary-indexes.html
https://dev.mysql.com/doc/refman/8.4/en/create-table-secondary-indexes.html

19 21 22 25 26 28 30 32 38 39 40 53 https://lln-mahjong-club.be/inc/files/Riichi-rules-2016-EN.pdf
https://lln-mahjong-club.be/inc/files/Riichi-rules-2016-EN.pdf

31 https://en.wikipedia.org/wiki/Japanese_mahjong_scoring_rules
https://en.wikipedia.org/wiki/Japanese_mahjong_scoring_rules

52 https://github.com/smly/mjai.app
https://github.com/smly/mjai.app